WOXSEN UNIVERSITY

B.Tech Computer Science and Engineering (AI/ML)

DATABASE MANAGEMENT SYSTEMS
PROJECT-BASED LEARNING (PBL) REPORT

Project No. 19

CAR RENTAL RESERVATION AND VEHICLE RETURN MANAGEMENT SYSTEM

Submitted by

B. Deekshan Raj
Roll No.: 25WU0102037
Class / Section: AIML RHINOS
First Year, B.Tech CSE (AI/ML)

Academic Year 2026-2027



ABSTRACT

Small and mid-size car-rental agencies commonly manage vehicles, bookings and billing through paper registers and spreadsheets, which leads to double-booking, disputed damage claims, billing errors and delayed management reports. This project presents the design and implementation of a Car Rental Reservation and Vehicle Return Management System built on a normalized MySQL 8.0 database. The conceptual design identifies twelve entities, and the relational schema is normalized to Third Normal Form with primary, foreign, unique and check constraints. Business rules that span several rows, such as preventing overlapping reservations, rejecting expired driving licences and updating vehicle status at pickup and return, are enforced through database triggers so that they hold regardless of the connecting application. The data layer is exposed through a Flask REST API in Python, which runs each multi-step workflow as a single transaction, and a React web interface supports vehicle and customer management, availability search, reservation, vehicle issue, return inspection, automatic invoicing and payments. Six reporting views provide fleet availability, overdue rentals, utilisation, damage, branch revenue and customer history. Tests of the billing logic, input validation and rule enforcement confirm that the system behaves as specified.

Keywords: car rental system, relational database, MySQL triggers, third normal form, REST API, invoice automation

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION 1

1.1 Background 1

1.2 Problem Statement 1

1.3 Objectives 1

1.4 Scope and Assumptions 2

1.5 Solution Strategy 2

CHAPTER 2: SYSTEM ANALYSIS AND REQUIREMENTS 3

2.1 Existing System 3

2.2 Proposed System 3

2.3 Functional Requirements 3

2.4 Non-Functional Requirements 4

2.5 Technology Stack 4

CHAPTER 3: CONCEPTUAL DESIGN 6

3.1 Entity Identification 6

3.2 Entity-Relationship Diagram 6

3.3 Relationships and Cardinalities 8

3.4 Business Process 8

CHAPTER 4: LOGICAL DESIGN 9

4.1 Relational Schema 9

4.2 Keys and Integrity Constraints 10

4.3 Functional Dependencies 10

4.4 Normalization up to 3NF 11

4.5 Data Dictionary 12

CHAPTER 5: PHYSICAL IMPLEMENTATION 15

5.1 Database Creation and DDL 15

5.2 Triggers 15

5.3 Sample Data 16

5.4 SQL Queries and Views 17

CHAPTER 6: APPLICATION IMPLEMENTATION 18

6.1 Architecture 18

6.2 Database Connectivity 18

6.3 Module-to-Table Mapping 19

6.4 CRUD Operations 20

6.5 Business-Rule Validation 20

6.6 Issue, Return and Invoice Workflow 21

6.7 Payments 22

6.8 Reports 22

6.9 Application Screens 22

CHAPTER 7: TESTING AND RESULTS 25

7.1 Test Strategy 25

7.2 Billing Test Cases 25

7.3 Input Validation Test Cases 25

7.4 Database Rule Test Cases 26

7.5 Results and Discussion 26

CHAPTER 8: CONCLUSION AND FUTURE WORK 28

8.1 Conclusion 28

8.2 Limitations 28

8.3 Future Work 28

REFERENCES 32

APPENDIX A: DATABASE SCRIPT 30

APPENDIX B: SOURCE CODE REFERENCE 35

LIST OF TABLES

Table 1. Project Objectives and Measures of Success 1

Table 2. Existing System versus Proposed System 3

Table 3. Functional Requirements 3

Table 4. Technology Stack 4

Table 5. Entities and Their Purpose 6

Table 6. Relationships and Cardinalities 8

Table 7. Logical Relational Schema 9

Table 8. Domain and Uniqueness Constraints 10

Table 9. Functional Dependencies and Normal Form 10

Table 10. Data Dictionary 12

Table 11. Sample Data Loaded 16

Table 12. Customer Table Excerpt (SELECT * FROM Customer) 16

Table 13. Reporting Views 17

Table 14. Module-to-Table Mapping 19

Table 15. CRUD Endpoints 20

Table 16. Business Rules and Where They Are Enforced 20

Table 17. Invoice Calculation Rules 21

Table 18. Worked Invoice Example 22

Table 19. Billing Test Cases 25

Table 20. Input Validation Test Cases 25

Table 21. Database Rule Test Cases 26

LIST OF FIGURES

Figure 1. Entity-Relationship Diagram of the Car Rental System 7

Figure 2. Reservation-to-Payment Process Flow 8

Figure 3. Three-Tier Architecture of the System 18

Figure 4. Landing Page of the Application 23

Figure 5. Reserve, Collect and Return Process 23

Figure 6. Branch Listing 24

Figure 7. Reservation Screen with Car Selection 24

□

CHAPTER 1: INTRODUCTION

1.1 Background

Car-rental agencies run on a small number of repeating events: a customer reserves a vehicle for a period, the vehicle is handed over after an inspection, it is used, it is returned and inspected again, and a bill is produced and settled. In small and mid-size agencies these events are still frequently recorded in paper registers, spreadsheets and phone confirmations kept separately at each branch. This approach cannot guarantee that the records of different branches agree with one another [1].

A database management system (DBMS) addresses this directly. A well-designed relational database stores each fact once, enforces rules on the data itself, and lets many users and programs work on the same records concurrently without corrupting them [1], [2]. This project applies those ideas to a realistic business domain, a multi-branch car-rental agency operating in Mumbai, Bengaluru and Pune, and builds a working system on top of the resulting database.

1.2 Problem Statement

Manual rental tracking fails in four concrete ways. First, the same vehicle can be promised to two customers for overlapping periods because there is no real-time availability check. Second, disputes about damage cannot be settled because there is no consistent record of the vehicle's condition at pickup and at return. Third, rental days, extra kilometres and late fees are calculated by hand and therefore inconsistently. Fourth, management cannot see branch-wise revenue or fleet utilisation without spending days compiling figures.

The problem addressed by this project is therefore to design and implement a centralised, normalized relational database and an application on top of it that (a) prevents double-booking and the hiring of vehicles to customers with expired licences, (b) records structured pickup and return inspections, (c) computes invoices automatically and consistently, and (d) produces management reports directly from the stored data.

1.3 Objectives

The objectives are stated so that each can be checked against a measurable outcome, as summarised in Table 1.

Table 1. Project Objectives and Measures of Success

No.	Objective	Measure of success
1	Identify entities, attributes and relationships through requirement analysis	12 entities, each with a primary key, documented in Chapter 3
2	Design a conceptual ER diagram covering all principal entities	One ER diagram with 12 entities and 13 relationships with cardinalities
3	Normalize the relational schema up to Third Normal Form (3NF)	Functional dependencies listed and verified for all 12 relations
4	Enforce business rules inside the database	3 triggers, 9 named CHECK constraints and UNIQUE keys on natural identifiers
5	Populate the database with realistic, testable sample data	Branches, categories, vehicles, customers and reservations loaded
6	Write joins, aggregates, nested subqueries and views for reports	6 reporting views and a nested-subquery example
7	Build an application that uses the database for all operations	REST API with CRUD and transactional workflows; web interface
8	Verify the system with positive and negative tests	Billing, validation and database-rule test cases with specific results

1.4 Scope and Assumptions

The system covers the complete rental life cycle for one agency with several branches: fleet and customer master data, availability search, reservation, vehicle issue with pickup inspection, return with return inspection and damage recording, automatic invoicing, payment recording and six management reports. The following assumptions are made:

- Vehicles are rented by the day. Each reservation is for exactly one vehicle and is priced by one rate plan that belongs to the vehicle's category.
- Each rate plan carries a daily rate, an extra-kilometre rate and a late fee per hour. Every rental includes 200 free kilometres per billable day.
- All amounts are in Indian rupees and tax is charged at a flat 18 percent on the sum of

all charges.

- A customer may pay an invoice in several instalments, using cash, card, UPI or net banking.
- Online payment gateways, GPS tracking, customer self-service log-in and multi-currency billing are outside the scope of this project.

1.5 Solution Strategy

The solution is database-centred. The relational schema is designed first, normalized to 3NF, and implemented in MySQL 8.0 [3] with primary keys, foreign keys, UNIQUE and CHECK constraints. Rules that need to compare a new row with other rows, which a CHECK constraint cannot do, are implemented as triggers. Reporting needs are met by views. Only after the data layer is complete is the application built: a Flask [4] REST API written in Python that talks to MySQL through the official connector [5], and a React [6] web interface that calls the API. Every multi-step workflow in the API runs inside a single database transaction, so a failure at any step leaves no partial data behind.

CHAPTER 2: SYSTEM ANALYSIS AND REQUIREMENTS

2.1 Existing System

In the existing system each branch keeps its own paper register or spreadsheet. Availability is checked by phoning other staff or by looking through the register, so there is no guarantee that a vehicle promised to one customer has not already been promised to another. The condition of a vehicle is described informally, if at all, so a scratch noticed at return cannot be shown to have been absent at pickup. Charges are computed by hand from the number of days, the distance driven and any delay, which makes two staff members likely to produce two different bills for the same rental. Licence validity is checked by eye, and reports require manual consolidation across branches.

2.2 Proposed System

The proposed system replaces the registers with one centralised, normalized MySQL database shared by all branches. Rules are enforced by the database itself, so they hold no matter which program connects to it. Table 2 contrasts the two systems.

Table 2. Existing System versus Proposed System

Aspect	Existing system	Proposed system
Record keeping	Paper registers or spreadsheets per branch	Centralised, normalized MySQL database
Availability	No real-time check; double-booking possible	Trigger-based overlap prevention and availability search
Vehicle condition	No systematic pickup and return record	Structured pickup and return inspections with fuel readings and damage records
Billing	Manual, error-prone calculation	Automatic invoice with extra-km, late fee, damage and 18% tax
Licence validity	Not enforced	Licence expiry checked at booking time
Reporting	Days of manual consolidation	Six reporting views available on demand

2.3 Functional Requirements

The functional requirements derived from the problem statement are listed in Table 3. Each requirement is traced to the module that implements it in Chapter 6.

Table 3. Functional Requirements

ID	Requirement	Module
FR1	Create, view, update and delete vehicles; status is one of AVAILABLE, RENTED, MAINTENANCE or RETIRED	Vehicles
FR2	Create, view, update and delete customers, including licence number and expiry	Customers
FR3	Provide branches, vehicle categories and rate plans as reference data	Reference data
FR4	Search vehicles available for a pickup and return period, filtered by category and branch	Availability
FR5	Create a reservation; reject overlapping bookings and customers with expired licences	Reservations
FR6	Cancel a reservation that has not yet been issued	Reservations
FR7	Issue a vehicle against a reservation with a pickup inspection and fuel	Issue and Return

	reading	
FR8	Record the return with a return inspection, fuel reading and any damages	Issue and Return
FR9	Generate the invoice automatically from days used, distance, delay and damage	Issue and Return
FR10	Record full or partial payments and track the outstanding balance	Invoices and Payments
FR11	Provide six management reports: fleet availability, overdue rentals, vehicle utilisation, damage, branch revenue and customer history	Reports
FR12	Show summary statistics (fleet size, branches, fleet kilometres, clients) on the landing page	Reference data

2.4 Non-Functional Requirements

- **Integrity:** business rules must hold regardless of the application that connects, so they are placed in the database wherever possible.
- **Consistency:** multi-step operations such as issuing or returning a vehicle must be atomic.
- **Security:** all SQL issued by the application uses parameterised queries; database credentials are read from an environment file and never hard-coded.
- **Accuracy:** monetary values are held as decimals, not floating-point numbers, so stored amounts match to the paisa.

2.5 Technology Stack

The technologies used are summarised in Table 4. Versions are those pinned in the project's dependency files.

Table 4. Technology Stack

Layer	Technology	Role
Database	MySQL 8.0	Tables, constraints, triggers and views
API framework	Python with Flask 3.0.3	REST endpoints, JSON responses, error handling
Database driver	mysql-connector-python 9.0.0	Connection, transactions, error codes
Supporting libraries	flask-cors 5.0.0, python-dotenv 1.0.1, gunicorn 22.0.0	Cross-origin access, settings from .env, production server
Web interface	React 19 with Vite 8	Single-page application and build tooling
Routing and UI	react-router-dom 7, gsap, lenis, motion, Phosphor icons	Navigation, animation and icons

CHAPTER 3: CONCEPTUAL DESIGN

3.1 Entity Identification

Requirement analysis identified twelve entities. They fall into three groups: reference entities that change rarely (Branch, VehicleCategory, RatePlan), master entities (Vehicle, Customer), and transaction entities that record what happens during a rental (Reservation, Rental, Inspection, FuelReading, Damage, Invoice, Payment). Table 5 describes each entity.

Table 5. Entities and Their Purpose

Entity	Purpose	Key attributes
Branch	Rental office location	branch_id, branch_name, city, address, phone
VehicleCategory	Hatchback, Sedan, SUV or Luxury class	category_id, category_name, seating_capacity, base_daily_rate
RatePlan	Priced plan for a category	rate_plan_id, plan_name, daily_rate, extra_km_rate, late_fee_per_hour
Vehicle	Individual fleet vehicle	vehicle_id, registration_no, model_name, odometer_km, status
Customer	Renter with licence details	customer_id, full_name, phone, license_number, license_expiry
Reservation	Advance booking of a vehicle	reservation_id, planned_pickup, planned_return, status
Rental	Actual pickup and return of a booked vehicle	rental_id, actual_pickup_dt, actual_return_dt, odometers
	Condition check at pickup or	inspection_id, inspection_type, odometer_reading,

Inspection	return	fuel_level_pct
FuelReading	Fuel level recorded for an inspection	fuel_reading_id, fuel_level_pct
Damage	Damage found at return and its repair cost	damage_id, description, repair_cost
Invoice	Bill for one completed rental	invoice_id, charge components, tax_amount, total_amount, status
Payment	Instalment paid against an invoice	payment_id, amount, payment_mode, payment_date

3.2 Entity-Relationship Diagram

Figure 1 shows the final approved ER diagram using crow's-foot notation. Each box lists the entity's primary key (PK), foreign keys (FK) and principal attributes, and every line is labelled with its cardinality. The diagram is drawn on a landscape page so that attribute names remain readable. Entity names, keys and relationships in it match the implemented relational schema in Chapter 4.

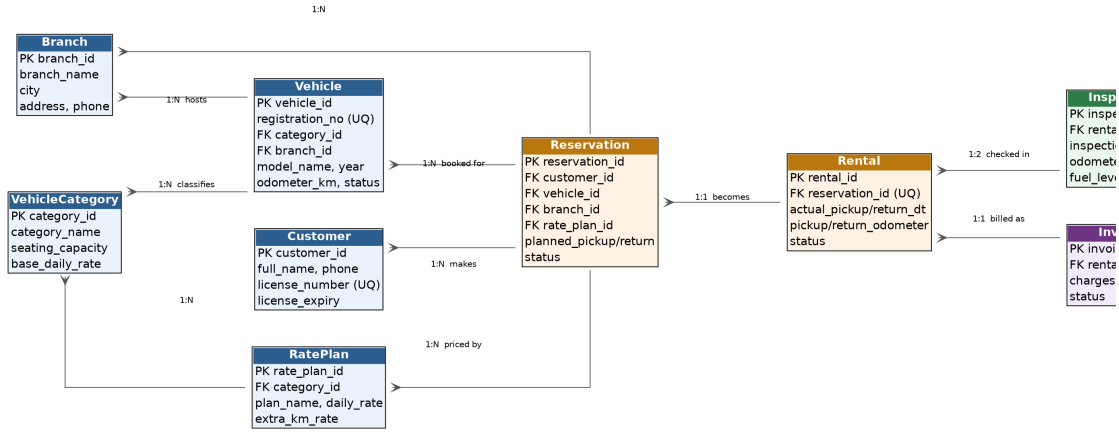


Figure 1. Entity-Relationship Diagram of the Car Rental System

3.3 Relationships and Cardinalities

The thirteen relationships in Figure 1 are described in Table 6. The one-to-one relationship between Reservation and Rental separates the promise (a booking) from the fact (a pickup that actually happened), so a booking can be cancelled without leaving an empty rental row. The one-to-two relationship between Rental and Inspection reflects that every completed rental has exactly one PICKUP and one RETURN inspection.

Table 6. Relationships and Cardinalities

Relationship	Entities	Cardinality
hosts	Branch to Vehicle	1:N
classifies	VehicleCategory to Vehicle	1:N
is priced by (category)	VehicleCategory to RatePlan	1:N
makes	Customer to Reservation	1:N
is booked for	Vehicle to Reservation	1:N
picks up at	Branch to Reservation	1:N
is priced by	RatePlan to Reservation	1:N
becomes	Reservation to Rental	1:1
is checked in	Rental to Inspection	1:2 (PICKUP, RETURN)
records fuel	Inspection to FuelReading	1:1
may record	Inspection to Damage	1:N
is billed as	Rental to Invoice	1:1
is settled by	Invoice to Payment	1:N

3.4 Business Process

Figure 2 shows how the data model follows one rental from search to payment. The first two

steps are guarded by triggers that block double-booking and expired licences. The vehicle's status changes automatically at issue and at return, so no staff member has to remember to update it.

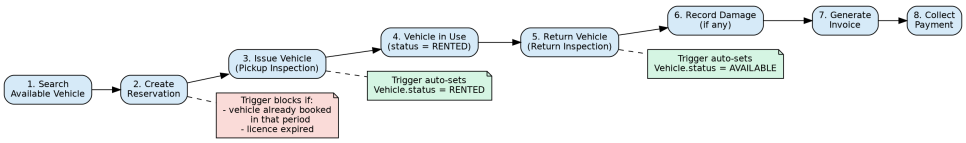


Figure 2. Reservation-to-Payment Process Flow

CHAPTER 4: LOGICAL DESIGN

4.1 Relational Schema

The ER diagram maps to twelve relations. Each entity becomes a relation, each one-to-many relationship becomes a foreign key on the "many" side, and each one-to-one relationship becomes a foreign key with a UNIQUE constraint. Table 7 gives the schema.

Table 7. Logical Relational Schema

Relation	Attributes	Primary key	Foreign keys
Branch	branch_id, branch_name, city, address, phone	branch_id	-
VehicleCategory	category_id, category_name, seating_capacity, base_daily_rate	category_id	-
RatePlan	rate_plan_id, category_id, plan_name, daily_rate, extra_km_rate, late_fee_per_hour	rate_plan_id	category_id
Vehicle	vehicle_id, registration_no, category_id, branch_id, model_name, manufacture_year, odometer_km, status	vehicle_id	category_id, branch_id
Customer	customer_id, full_name, phone, email, license_number, license_expiry, address	customer_id	-
Reservation	reservation_id, customer_id, vehicle_id, branch_id, rate_plan_id, booking_date, planned_pickup, planned_return, status	reservation_id	customer_id, vehicle_id, branch_id, rate_plan_id
Rental	rental_id, reservation_id, actual_pickup_dt, actual_return_dt, pickup_odometer, return_odometer, status	rental_id	reservation_id (UNIQUE)
Inspection	inspection_id, rental_id, inspection_type, odometer_reading, fuel_level_pct, condition_notes, inspected_by	inspection_id	rental_id
FuelReading	fuel_reading_id, inspection_id, fuel_level_pct	fuel_reading_id	inspection_id (UNIQUE)
Damage	damage_id, inspection_id, description, repair_cost	damage_id	inspection_id
Invoice	invoice_id, rental_id, rental_charges, extra_km_charges, late_fee, damage_charges, tax_amount, total_amount, status	invoice_id	rental_id (UNIQUE)
Payment	payment_id, invoice_id, amount, payment_mode, payment_date	payment_id	invoice_id

4.2 Keys and Integrity Constraints

Every relation has a surrogate integer primary key, and foreign keys provide referential integrity, so deleting a vehicle or customer referenced by a reservation is refused. Domain and uniqueness rules are listed in Table 8.

Table 8. Domain and Uniqueness Constraints

Constraint	Relation	Rule enforced
UNIQUE registration_no	Vehicle	No two vehicles share a registration number
UNIQUE phone, email, license_number	Customer	A customer is registered once
UNIQUE reservation_id	Rental	A reservation becomes at most one rental
UNIQUE rental_id	Invoice	A rental is invoiced at most once
UNIQUE inspection_id	FuelReading	One fuel reading per inspection
uq_inspection_type_per_rental	Inspection	One PICKUP and one RETURN inspection per rental
chk_vehicle_odometer	Vehicle	Odometer cannot be negative
chk_vehicle_year	Vehicle	Manufacture year between 1990 and 2100

chk_res_dates	Reservation	Planned return is after planned pickup
chk_rental_dates	Rental	Actual return is after actual pickup
chk_rental_odometer	Rental	Return odometer is not lower than pickup odometer
chk_inspection_fuel, chk_fuel_pct	Inspection, FuelReading	Fuel level between 0 and 100 percent
chk_damage_cost	Damage	Repair cost cannot be negative
chk_payment_amount	Payment	Payment amount is greater than zero

4.3 Functional Dependencies

The important functional dependencies of each relation are listed in Table 9. In every relation the determinant is the primary key (or, for the UNIQUE alternatives, a candidate key), which is the condition for Boyce-Codd Normal Form and therefore also for 3NF [1], [2].

Table 9. Functional Dependencies and Normal Form

Relation	Functional dependency	Normal form
Branch	branch_id → branch_name, city, address, phone	3NF
VehicleCategory	category_id → category_name, seating_capacity, base_daily_rate	3NF
RatePlan	rate_plan_id → category_id, plan_name, daily_rate, extra_km_rate, late_fee_per_hour	3NF
Vehicle	vehicle_id → registration_no, category_id, branch_id, model_name, manufacture_year, odometer_km, status	3NF
Customer	customer_id → full_name, phone, email, license_number, license_expiry, address	3NF
Reservation	reservation_id → customer_id, vehicle_id, branch_id, rate_plan_id, booking_date, planned_pickup, planned_return, status	3NF
Rental	rental_id → reservation_id, actual_pickup_dt, actual_return_dt, pickup_odometer, return_odometer, status	3NF
Inspection	(rental_id, inspection_type) → inspection_id, odometer_reading, fuel_level_pct, condition_notes, inspected_by	3NF
FuelReading	fuel_reading_id → inspection_id, fuel_level_pct	3NF
Damage	damage_id → inspection_id, description, repair_cost	3NF
Invoice	invoice_id → rental_id, charge components, tax_amount, total_amount, status	3NF
Payment	payment_id → invoice_id, amount, payment_mode, payment_date	3NF

4.4 Normalization up to 3NF

First normal form. Every attribute holds a single atomic value and every relation has a primary key. The one place where a repeating group could have appeared was the pickup and return inspection of a rental. Storing both inspections in the Rental row would require two copies of each inspection attribute, so they are held as separate rows of Inspection, distinguished by inspection_type.

Second normal form. All primary keys are single-column surrogate keys, so no non-key attribute can depend on part of a key. Inspection has the natural key (rental_id, inspection_type), but all of its other attributes depend on the whole of that key.

Third normal form. No non-key attribute depends on another non-key attribute. Several design decisions were taken to achieve this:

- VehicleCategory is separated from Vehicle. Seating capacity and base rate depend on the category, not on the individual vehicle, and keeping them in Vehicle would repeat them for every car and cause update anomalies.
- RatePlan is separated from VehicleCategory so that a category can have several priced plans with their own extra-kilometre and late-fee rates.
- Reservation stores its own branch_id. This is the branch where the vehicle is picked up, which may differ from the vehicle's home branch, so it is not derivable from Vehicle.
- Invoice charges are split into rental, extra-kilometre, late-fee and damage components, so that each can be traced to the rule that produced it.

The Invoice relation stores tax_amount and total_amount although they could be recomputed from the components. This is a deliberate, controlled redundancy: an issued invoice is a legal

record and must not change if a rate plan is edited later. The values are written once, in the same transaction that creates the invoice, and never updated.

4.5 Data Dictionary

Table 10 is the data dictionary. Types are those used in the DDL of Appendix A. Monetary columns are DECIMAL so that rounding is exact.

Table 10. Data Dictionary

Relation	Column	Type	Constraint / description
Branch	branch_id	INT AUTO_INCREMENT	PK
	branch_name	VARCHAR(50)	NOT NULL
	city	VARCHAR(40)	NOT NULL
	address, phone	VARCHAR(120), VARCHAR(15)	Optional contact details
VehicleCategory	category_id	INT AUTO_INCREMENT	PK
	category_name	VARCHAR(30)	NOT NULL
	seating_capacity	INT	NOT NULL
	base_daily_rate	DECIMAL(10,2)	NOT NULL, listing rate
RatePlan	rate_plan_id	INT AUTO_INCREMENT	PK
	category_id	INT	FK to VehicleCategory
	plan_name	VARCHAR(40)	NOT NULL
	daily_rate	DECIMAL(10,2)	NOT NULL
	extra_km_rate	DECIMAL(8,2)	NOT NULL, per km beyond free km
	late_fee_per_hour	DECIMAL(8,2)	NOT NULL
Vehicle	vehicle_id	INT AUTO_INCREMENT	PK
	registration_no	VARCHAR(15)	UNIQUE, NOT NULL
	category_id, branch_id	INT	FK to VehicleCategory, Branch
	model_name	VARCHAR(40)	NOT NULL
	manufacture_year	INT	CHECK 1990 to 2100
	odometer_km	INT	CHECK >= 0
	status	ENUM	AVAILABLE, RENTED, MAINTENANCE, RETIRED
Customer	customer_id	INT AUTO_INCREMENT	PK
	full_name	VARCHAR(60)	NOT NULL
	phone	VARCHAR(15)	UNIQUE, digits only
	email	VARCHAR(80)	UNIQUE, optional
	license_number	VARCHAR(20)	UNIQUE, NOT NULL
	license_expiry	DATE	NOT NULL
	address	VARCHAR(120)	Optional
Reservation	reservation_id	INT AUTO_INCREMENT	PK
	customer_id, vehicle_id, branch_id, rate_plan_id	INT	FK to Customer, Vehicle, Branch, RatePlan
	booking_date	DATETIME	Default current time
	planned_pickup, planned_return	DATETIME	CHECK return > pickup
	status	ENUM	BOOKED, ISSUED, COMPLETED, CANCELLED
Rental	rental_id	INT AUTO_INCREMENT	PK
	reservation_id	INT	FK, UNIQUE
	actual_pickup_dt, actual_return_dt	DATETIME	CHECK return > pickup
	pickup_odometer, return_odometer	INT	CHECK return >= pickup
	status	ENUM	Open, overdue or returned
Inspection	inspection_id	INT AUTO_INCREMENT	PK
	rental_id	INT	FK; UNIQUE with inspection_type
	inspection_type	ENUM	PICKUP, RETURN

	odometer_reading	INT	NOT NULL
	fuel_level_pct	INT	CHECK 0 to 100
	condition_notes, inspected_by	VARCHAR(200), VARCHAR(40)	Optional
FuelReading	fuel_reading_id	INT AUTO_INCREMENT	PK
	inspection_id	INT	FK, UNIQUE
	fuel_level_pct	INT	CHECK 0 to 100
Damage	damage_id	INT AUTO_INCREMENT	PK
	inspection_id	INT	FK to Inspection
	description	VARCHAR(200)	NOT NULL
	repair_cost	DECIMAL(10,2)	CHECK >= 0
Invoice	invoice_id	INT AUTO_INCREMENT	PK
	rental_id	INT	FK, UNIQUE
	rental_charges, extra_km_charges, late_fee, damage_charges	DECIMAL(12,2)	Charge components
	tax_amount, total_amount	DECIMAL(12,2)	Stored at billing time
	status	ENUM	UNPAID, PARTIAL, PAID
Payment	payment_id	INT AUTO_INCREMENT	PK
	invoice_id	INT	FK to Invoice
	amount	DECIMAL(12,2)	CHECK > 0
	payment_mode	ENUM	CASH, CARD, UPI, NETBANKING
	payment_date	DATETIME	Default current time

CHAPTER 5: PHYSICAL IMPLEMENTATION

5.1 Database Creation and DDL

The database `car_rental_db` is created in MySQL 8.0 using InnoDB tables, which provide foreign keys and transactions. MySQL 8.0 enforces CHECK constraints [3], so single-row rules are declared with the table. The following excerpt shows the representative DDL for the Vehicle and Reservation tables; the complete script is in Appendix A.

```
CREATE TABLE Vehicle (
 vehicle_id       INT AUTO_INCREMENT PRIMARY KEY,
 registration_no  VARCHAR(15) NOT NULL UNIQUE,
 category_id      INT NOT NULL,
 branch_id        INT NOT NULL,
 model_name       VARCHAR(40) NOT NULL,
 manufacture_year INT NOT NULL,
 odometer_km      INT NOT NULL DEFAULT 0,
 status ENUM('AVAILABLE','RENTED','MAINTENANCE','RETIRED')
        NOT NULL DEFAULT 'AVAILABLE',
 CONSTRAINT chk_vehicle_odometer CHECK (odometer_km >= 0),
 CONSTRAINT chk_vehicle_year CHECK (manufacture_year BETWEEN 1990 AND
2100),
 FOREIGN KEY (category_id) REFERENCES VehicleCategory(category_id),
 FOREIGN KEY (branch_id)   REFERENCES Branch(branch_id)
) ENGINE=InnoDB;


CREATE TABLE Reservation (
 reservation_id INT AUTO_INCREMENT PRIMARY KEY,
 customer_id INT NOT NULL, vehicle_id INT NOT NULL,
 branch_id INT NOT NULL,   rate_plan_id INT NOT NULL,
 booking_date   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
 planned_pickup DATETIME NOT NULL,
 planned_return DATETIME NOT NULL,
 status ENUM('BOOKED','ISSUED','COMPLETED','CANCELLED')
        NOT NULL DEFAULT 'BOOKED',
 CONSTRAINT chk_res_dates CHECK (planned_return > planned_pickup),
 FOREIGN KEY (customer_id)  REFERENCES Customer(customer_id),
 FOREIGN KEY (vehicle_id)   REFERENCES Vehicle(vehicle_id),
 FOREIGN KEY (branch_id)    REFERENCES Branch(branch_id),
 FOREIGN KEY (rate_plan_id) REFERENCES RatePlan(rate_plan_id)
) ENGINE=InnoDB;
```

5.2 Triggers

A CHECK constraint can only inspect the row being written. Two of the business rules, no overlapping reservation for a vehicle and no reservation for a customer whose licence has expired, need to compare the new row with other rows or with another table. These are implemented as BEFORE INSERT triggers on Reservation that raise SQLSTATE 45000 with a readable message when a rule is violated. A third trigger, an AFTER UPDATE trigger on Rental, changes the vehicle's status when a pickup or a return is recorded. Because the triggers live in the database, they apply to every client, not only to this application. The overlap trigger is shown below.

```
CREATE TRIGGER trg_no_overlap_reservation
BEFORE INSERT ON Reservation FOR EACH ROW
BEGIN
 IF EXISTS (SELECT 1 FROM Reservation r
            WHERE r.vehicle_id = NEW.vehicle_id
              AND r.status IN ('BOOKED','ISSUED')
              AND NEW.planned_pickup < r.planned_return
              AND NEW.planned_return > r.planned_pickup) THEN
   SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT =
     'Overlapping reservation exists for this vehicle in the selected
period.';
 END IF;
END;
```

The licence trigger, `trg_valid_license`, rejects a reservation whose planned pickup date is later than the customer's licence expiry date, with the message "Customer driving licence is expired before the pickup date." The status trigger, `trg_vehicle_status_on_pickup`, sets Vehicle.status to RENTED when actual_pickup_dt changes from NULL to a value, and to AVAILABLE, with the new odometer reading, when actual_return_dt is filled in.

5.3 Sample Data

The database was populated with realistic data so that every rule and report could be exercised. Table 11 gives the number of rows loaded in the main tables, and Table 12 shows an excerpt of the Customer table. One customer's licence in the sample data has already lapsed, and one rental (Rental 2, Ananya Rao, Hyundai Creta) is still outstanding past its planned return, which is what the Overdue Rentals report displays.

Table 11. Sample Data Loaded

Table	Rows loaded
Branch	3
VehicleCategory	4
Vehicle	6
Customer	4
Reservation	3

Table 12. Customer Table Excerpt (SELECT * FROM Customer)

customer_id	full_name	license_expiry
1	Rohan Mehta	2027-05-10
2	Ananya Rao	2026-11-20
3	Karan Verma	2025-01-15

5.4 SQL Queries and Views

Six views provide the management reports. Each is built from joins and aggregate functions over the base tables and is read-only from the application's point of view. Table 13 lists them.

Table 13. Reporting Views

View	Report	Question it answers
vw_fleet_availability	Fleet Availability	Which vehicles are available, rented or unavailable at each branch?
vw_overdue_rentals	Overdue Rentals	Which vehicles are past their planned return and not yet back?
vw_vehicle_utilization	Vehicle Utilization	How heavily is each vehicle used?
vw_damage_report	Damage Report	What damage was recorded, on which vehicle, and at what repair cost?

vw_branch_revenue	Branch Revenue	How much does each branch invoice and collect?
vw_customer_history	Customer History	What has each customer rented?

Beyond the views, a nested subquery finds the customers who paid more than the average invoice. The inner query computes the average over all invoices once, and the outer query joins four tables to list the customers whose invoice total exceeds it.

```
SELECT DISTINCT c.full_name, inv.total_amount
FROM Customer c
JOIN Reservation r ON r.customer_id = c.customer_id
JOIN Rental rt     ON rt.reservation_id = r.reservation_id
JOIN Invoice inv   ON inv.rental_id = rt.rental_id
WHERE inv.total_amount > (SELECT AVG(total_amount) FROM Invoice);
```

CHAPTER 6: APPLICATION IMPLEMENTATION

6.1 Architecture

The system follows a three-tier architecture, shown in Figure 3. The presentation tier is a React single-page application; the application tier is a Flask REST API that exposes the database through JSON endpoints under `/api`; the data tier is the MySQL database with its constraints, triggers and views. The first prototype of this project used a desktop Tkinter interface. It was replaced by a web interface so that several branches can use the system at once, and the database access logic was rewritten as a reusable API; the database design was not affected by this change.

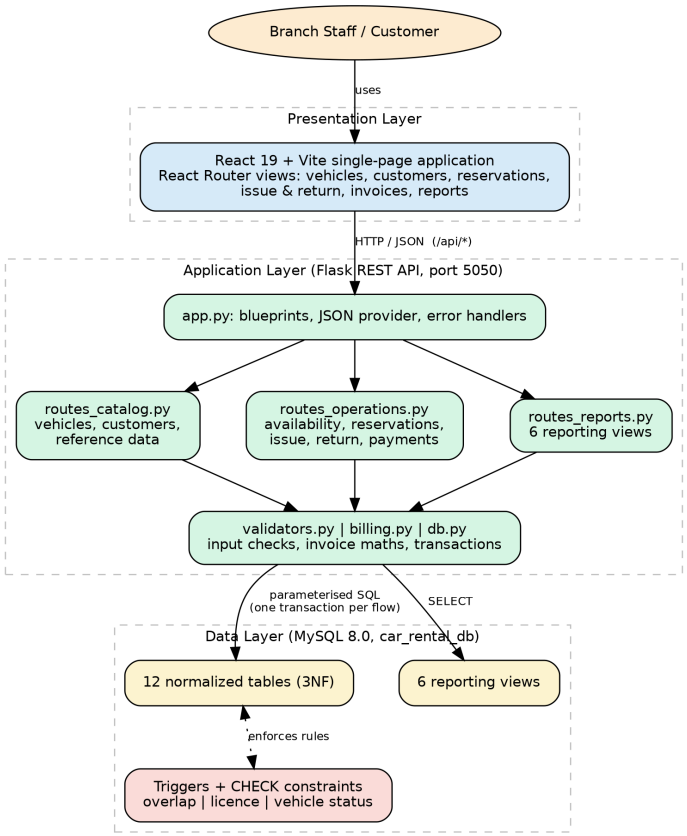


Figure 3. Three-Tier Architecture of the System

6.2 Database Connectivity

All database access goes through one module, `db.py`. Connection settings (host, port, user, password and database name) are read from an environment file with python-dotenv, so no credentials appear in the source. The module provides a `transaction()` context manager that opens one connection, disables autocommit, commits when the block finishes and rolls back on any exception. Using one connection and one transaction per workflow means that `cursor.lastrowid` is always valid and that a failure at any step undoes every earlier step.

```
@contextmanager
def transaction():
    conn = get_connection()
```

```
    conn.autocommit = False
    cursor = conn.cursor(dictionary=True)
    try:
        yield cursor
        conn.commit()
    except mysql.connector.Error as e:
        conn.rollback()
        raise translate_mysql_error(e)
    except Exception:
        conn.rollback()
        raise
    finally:
        cursor.close()
        conn.close()
```

Every query passes user-supplied values as parameters (`%s` placeholders) and never builds SQL by string concatenation. The only dynamic SQL is the report endpoint, which selects the view name from a fixed whitelist of six names. Database errors are translated into clean API errors: the message of a trigger (SQLSTATE 45000) is returned verbatim with HTTP status 409, duplicate-key and CHECK violations are mapped to friendly messages naming the offending field, and an unreachable database returns status 503.

6.3 Module-to-Table Mapping

Table 14 shows which tables each application module reads and writes.

Table 14. Module-to-Table Mapping

Module	Tables read	Tables written
Reference data and statistics	VehicleCategory, Branch, RatePlan, Vehicle, Customer	-
Vehicles	Vehicle, VehicleCategory, Branch, RatePlan	Vehicle
Customers	Customer, Reservation (count)	Customer
Availability search	Vehicle, VehicleCategory, Branch, RatePlan, Reservation	-
Reservations	Reservation, Customer, Vehicle, Branch, RatePlan, Rental	Reservation
Issue vehicle	Reservation, Vehicle	Rental, Inspection, FuelReading, Reservation; Vehicle via trigger
Return and invoice	Rental, Reservation, RatePlan	Rental, Inspection, FuelReading, Damage, Invoice, Reservation; Vehicle via trigger
Payments	Invoice, Payment	Payment, Invoice
Reports	Six views over the base tables	-

6.4 CRUD Operations

Vehicles and customers have full create, read, update and delete operations; reservations are created, read and cancelled. Table 15 lists the endpoints.

Table 15. CRUD Endpoints

Operation	Method and path	Table	Notes
Create	POST /api/vehicles	Vehicle	Registration number upper-cased; returns the new row
Read	GET /api/vehicles, /api/vehicles/{id}	Vehicle	Joined with category, branch and lowest daily rate
Update	PUT /api/vehicles/{id}	Vehicle	Row locked with FOR UPDATE; RENTED status cannot be set or cleared manually
Delete	DELETE /api/vehicles/{id}	Vehicle	Refused by foreign keys if referenced
Create	POST /api/customers	Customer	Phone, email and licence validated
Read	GET /api/customers, /api/customers/{id}	Customer	Includes license_expired flag and reservation count
Update	PUT /api/customers/{id}	Customer	Full replacement of editable fields
Delete	DELETE /api/customers/{id}	Customer	Refused by foreign keys if referenced
Create	POST /api/reservations	Reservation	Triggers enforce overlap and licence rules
Read	GET /api/reservations, /api/reservations/{id}	Reservation	Optional status filter
Update	POST /api/reservations/{id}/cancel	Reservation	Only BOOKED reservations can be cancelled

6.5 Business-Rule Validation

Validation is layered. The application rejects bad input early with a precise message, the transaction checks the current state of the affected rows while holding a row lock, and the database constraints and triggers remain the final guarantee. Table 16 shows where each rule is enforced.

Table 16. Business Rules and Where They Are Enforced

Business rule	Database layer	Application layer
No overlapping reservation for a vehicle	trg_no_overlap_reservation	Availability search excludes overlapping vehicles
Licence must be valid at pickup	trg_valid_license	Customer list flags expired licences
Return after pickup	chk_res_dates, chk_rental_dates	Checked before the database call
Return odometer not below pickup	chk_rental_odometer	Compared with the recorded pickup reading
Fuel level 0 to 100 percent	chk_inspection_fuel, chk_fuel_pct	Integer validator with range
Repair cost not negative	chk_damage_cost	Money validator
Payment positive and not above balance	chk_payment_amount	Amount compared with outstanding balance
Vehicle status follows pickup and return	trg_vehicle_status_on_pickup	Manual change to or from RENTED refused
Rate plan matches vehicle category	Foreign key only	Explicit category comparison
Maintenance or retired vehicles cannot be booked	Status values	Blocked statuses checked at reservation and issue
Valid reservation life cycle	Status ENUM	Only BOOKED can be issued or cancelled
Unique identifiers	UNIQUE keys	Duplicate-key errors mapped to messages

6.6 Issue, Return and Invoice Workflow

Issuing a vehicle runs in one transaction. The reservation row is locked, and the application confirms that it is BOOKED, that the vehicle is not under maintenance, retired or already rented, and that the pickup odometer is not below the vehicle's recorded value. It then inserts the Rental row, updates it with the pickup time and odometer (which fires the status trigger and sets the vehicle to RENTED), inserts the PICKUP inspection and its fuel reading, and marks the reservation ISSUED.

Returning a vehicle follows the same pattern. After locking the rental and checking that it was issued and not already returned, the application updates the return time and odometer (the trigger sets the vehicle to AVAILABLE), inserts the RETURN inspection and fuel reading, inserts any damage rows, computes the invoice, inserts the Invoice row and marks the reservation COMPLETED. Invoice arithmetic is isolated in a single function that uses `Decimal` values and rounds half-up to two places. Table 17 gives the rules.

Table 17. Invoice Calculation Rules

Component	Rule
Days used	max(1, (actual return − actual pickup).days + 1)
Rental charges	daily_rate × days used
Free kilometres	200 × days used
Extra-kilometre charges	max(0, kilometres driven − free kilometres) × extra_km_rate
Late fee	whole hours after planned return × late_fee_per_hour
Damage charges	sum of repair costs recorded at the return inspection
Tax	18% of (rental + extra-km + late fee + damage)
Total	subtotal + tax

Table 18 works through one rental that includes extra kilometres, a late return and damage, using test rates of 3,000 per day, 12 per extra kilometre and 250 per late hour. The vehicle is picked up on 2026-09-01 at 09:00 with the odometer at 1,000 km, was due back on 2026-09-02 at 10:00, and is returned on 2026-09-02 at 20:00 at 1,900 km with 1,500 of damage. The figures in the table are the output of the billing function.

Table 18. Worked Invoice Example

Step	Calculation	Amount
Days used	Elapsed 1 day 11 h, so 1 + 1	2 days
Rental charges	2 × 3,000	6,000.00
Extra kilometres	900 km driven − 400 free = 500 km; 500 × 12	6,000.00
Late fee	10 whole hours late; 10 × 250	2,500.00
Damage charges	Repair cost recorded at return	1,500.00
Subtotal	6,000 + 6,000 + 2,500 + 1,500	16,000.00
Tax at 18%	16,000 × 0.18	2,880.00
Total	16,000 + 2,880	18,880.00

6.7 Payments

An invoice may be settled in instalments. When a payment is recorded, the invoice row is locked, the amount already paid is summed, and the new amount is rejected if it is zero or exceeds the outstanding balance. After the payment row is inserted, the invoice status becomes PARTIAL or PAID depending on whether the balance has been cleared. Invoice listings show the amount paid and the balance computed from the Payment table, and can be filtered to unpaid invoices.

6.8 Reports

The six views of Table 13 are served by a single report endpoint that selects the view from a whitelist, so the view name never comes from user input. The Customer History report accepts an optional customer filter, which is passed as a bound parameter.

6.9 Application Screens

Figure 4 to Figure 7 show the web interface: the landing page, the three-step rental process, the branch listing and the reservation screen in which a car is chosen before the client, dates and plan are entered.

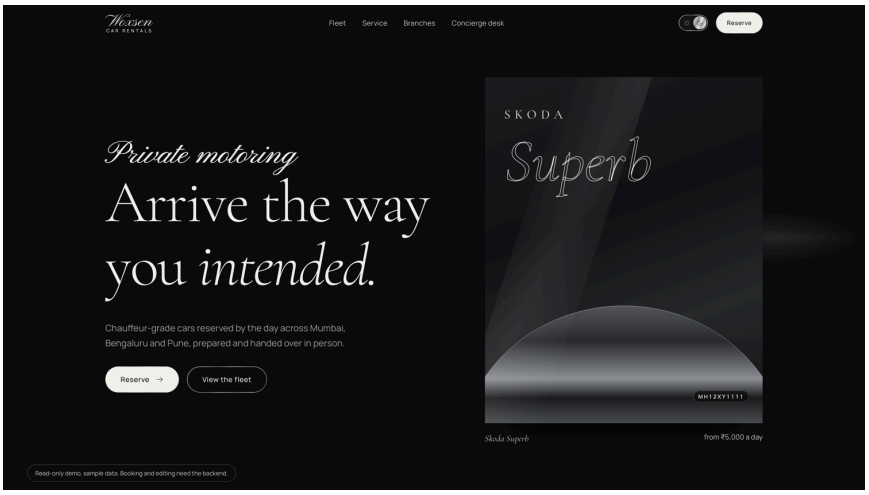


Figure 4. Landing Page of the Application



Figure 5. Reserve, Collect and Return Process



Figure 6. Branch Listing

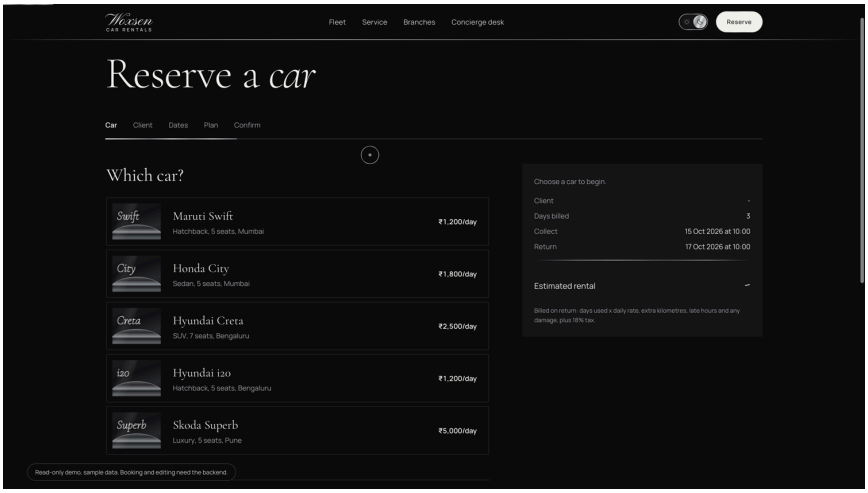


Figure 7. Reservation Screen with Car Selection

CHAPTER 7: TESTING AND RESULTS

7.1 Test Strategy

Testing covers three levels. Billing tests check the invoice function against hand-computed totals. Validation tests check that each input parser accepts valid values and rejects invalid ones with the intended message. Database-rule tests check that the constraints and triggers refuse invalid data. Both positive and negative cases are included. The billing and validation tests were run directly against the application's `billing.py` and `validators.py` modules, and the Actual column records the values they returned.

7.2 Billing Test Cases

Table 19 lists five billing cases at rates of 3,000 per day, 12 per extra kilometre and 250 per late hour. Expected totals were computed by hand from the rules in Table 17 before running the function.

Table 19. Billing Test Cases

ID	Scenario	Expected total	Actual total	Status
B1	Same-day rental, on time, 150 km driven (within 200 free km)	3,540.00	3,540.00	Pass
B2	3 billable days, 800 km driven (600 free), on time	13,452.00	13,452.00	Pass
B3	One day, returned 5 h 30 min late (5 whole hours charged), 100 km	5,015.00	5,015.00	Pass
B4	One day, on time, 100 km, damage of 4,500	8,850.00	8,850.00	Pass
B5	Extra km, 10 h late and damage of 1,500 (see worked example)	18,880.00	18,880.00	Pass

7.3 Input Validation Test Cases

Table 20 lists eleven validation cases, nine negative and two positive.

Table 20. Input Validation Test Cases

ID	Input	Expected result	Actual result	Status
V1	Phone "98a76"	Rejected: digits only	Phone must contain digits only.	Pass
V2	Email "abc@"	Rejected: invalid email	Email address is not valid.	Pass
V3	Odometer -5	Rejected: below minimum	Odometer must be at least 0.	Pass
V4	Payment amount 0	Rejected: must be positive	Amount must be greater than zero.	Pass
V5	Pickup "01/09/2026"	Rejected: wrong date format	Pickup must be a date and time (YYYY-MM-DD HH:MM).	Pass
V6	Vehicle status "BROKEN"	Rejected: not an allowed status	Status must be one of: AVAILABLE, RENTED, MAINTENANCE, RETIRED.	Pass
V7	Fuel level 120	Rejected: above 100	Fuel level must be at most 100.	Pass
V8	Registration "MH12AB1234567890X" (17 characters)	Rejected: over 15 characters	Registration number must be at most 15 characters.	Pass
V9	Phone "9876543210"	Accepted	Accepted: 9876543210	Pass
V10	Pickup "2026-09-01T10:30"	Accepted	Accepted: 2026-09-01 10:30:00	Pass
V11	Repair cost -10	Rejected: cannot be negative	Repair cost cannot be negative.	Pass

7.4 Database Rule Test Cases

Table 21 lists the negative tests that exercise the database constraints and triggers directly with SQL. Their expected results follow from the design in Chapters 4 and 5; the actual outcome of each is recorded when the test is run on the demonstration machine.

Table 21. Database Rule Test Cases

ID	Test	Expected result	Actual result	Status
DB1	INSERT a reservation for vehicle 1 overlapping an existing BOOKED reservation	Rejected: "Overlapping reservation exists for this vehicle in the selected period."	To be recorded	Pending
DB2	Set customer 1 licence expiry to 2020-01-01, then reserve for 2027-02-01	Rejected: "Customer driving licence is expired before the pickup date."	To be recorded	Pending
DB3	INSERT a vehicle with a registration number that already exists	Rejected by UNIQUE key	To be recorded	Pending
DB4	UPDATE a rental with a return odometer lower than the pickup odometer	Rejected by chk_rental_odometer	To be recorded	Pending
DB5	INSERT a second invoice for a rental that is already invoiced	Rejected by UNIQUE rental_id	To be recorded	Pending
DB6	UPDATE Rental to record a pickup, then SELECT the vehicle	Vehicle status becomes RENTED	To be recorded	Pending

7.5 Results and Discussion

All five billing cases and all eleven validation cases produced the expected result, so the invoice arithmetic and the input parsers behave as specified. Two observations from the billing tests are worth recording. First, the day count is based on the elapsed time in whole days plus one, so a rental that runs for exactly 24 hours is billed as two days, while a same-day rental is billed as one. This follows the pricing rule adopted for the project, in which the day of pickup is always billable. Second, the late fee counts only whole hours after the planned return, so a delay of 5 hours 30 minutes is charged as 5 hours.

Because the overlap and licence rules are implemented as triggers, the same protection applies whether a reservation arrives through the API or through a direct SQL statement. Edge cases such as concurrent bookings of the same vehicle and performance with large data volumes have not yet been tested and are listed as future work.

CHAPTER 8: CONCLUSION AND FUTURE WORK

8.1 Conclusion

This project designed and implemented a Car Rental Reservation and Vehicle Return Management System around a normalized MySQL database. The twelve-entity design was verified to be in Third Normal Form, and the rules that matter to the business, no double-

booking, no expired licences, valid readings and amounts, and automatic vehicle status, are enforced inside the database through constraints and triggers. A Flask REST API runs every multi-step workflow as a single transaction and a React interface lets staff manage the fleet, take reservations, issue and receive vehicles, generate invoices, record payments and view six management reports. The objectives set in Chapter 1 were met.

8.2 Limitations

- The system has no user authentication or role-based access. Cross-origin access is open by default and should be restricted through the CORS_ORIGINS setting in deployment.
- Concurrent bookings and large data volumes have not been load-tested.
- Rates are fixed per rate plan; there is no seasonal or dynamic pricing.

8.3 Future Work

- Add staff log-in with roles for branch staff and managers.
- Integrate an online payment gateway and send booking and return reminders to customers.
- Add indexes based on measured query plans and run concurrency tests on the reservation trigger.
- Record GPS-based distance and support one-way rentals between branches.

REFERENCES

[1] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Hoboken, NJ, USA: Pearson, 2016.

[2] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[3] Oracle Corporation, "MySQL 8.0 Reference Manual," Oracle, 2024. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

[4] Pallets, "Flask Documentation," 2024. [Online]. Available: https://flask.palletsprojects.com/

[5] Oracle Corporation, "MySQL Connector/Python Developer Guide," 2024. [Online]. Available: https://dev.mysql.com/doc/connector-python/en/

[6] Meta Platforms, Inc., "React Documentation," 2024. [Online]. Available: https://react.dev/

APPENDIX A: DATABASE SCRIPT

```
-- ============================================================
-- Car Rental Reservation and Vehicle Return Management System
-- Database: car_rental_db (MySQL 8.0)
-- ============================================================
CREATE DATABASE IF NOT EXISTS car_rental_db;
USE car_rental_db;


-- ---------------------------- reference tables
CREATE TABLE Branch (
 branch_id   INT AUTO_INCREMENT PRIMARY KEY,
 branch_name VARCHAR(50) NOT NULL,
 city        VARCHAR(40) NOT NULL,
 address     VARCHAR(120),
 phone       VARCHAR(15)
) ENGINE=InnoDB;


CREATE TABLE VehicleCategory (
 category_id      INT AUTO_INCREMENT PRIMARY KEY,
 category_name    VARCHAR(30) NOT NULL,
 seating_capacity INT NOT NULL,
 base_daily_rate  DECIMAL(10,2) NOT NULL
```

```
) ENGINE=InnoDB;


CREATE TABLE RatePlan (
 rate_plan_id      INT AUTO_INCREMENT PRIMARY KEY,
 category_id       INT NOT NULL,
 plan_name         VARCHAR(40) NOT NULL,
 daily_rate        DECIMAL(10,2) NOT NULL,
 extra_km_rate     DECIMAL(8,2) NOT NULL,
 late_fee_per_hour DECIMAL(8,2) NOT NULL,
 FOREIGN KEY (category_id) REFERENCES VehicleCategory(category_id)
) ENGINE=InnoDB;


-- ---------------------------- master tables
CREATE TABLE Vehicle (
 vehicle_id       INT AUTO_INCREMENT PRIMARY KEY,
 registration_no  VARCHAR(15) NOT NULL UNIQUE,
 category_id      INT NOT NULL,
 branch_id        INT NOT NULL,
 model_name       VARCHAR(40) NOT NULL,
 manufacture_year INT NOT NULL,
 odometer_km      INT NOT NULL DEFAULT 0,
 status ENUM('AVAILABLE','RENTED','MAINTENANCE','RETIRED')
        NOT NULL DEFAULT 'AVAILABLE',
 CONSTRAINT chk_vehicle_odometer CHECK (odometer_km >= 0),
 CONSTRAINT chk_vehicle_year CHECK (manufacture_year BETWEEN 1990 AND
2100),
 FOREIGN KEY (category_id) REFERENCES VehicleCategory(category_id),
 FOREIGN KEY (branch_id)   REFERENCES Branch(branch_id)
) ENGINE=InnoDB;


CREATE TABLE Customer (
 customer_id    INT AUTO_INCREMENT PRIMARY KEY,
 full_name      VARCHAR(60) NOT NULL,
 phone          VARCHAR(15) NOT NULL UNIQUE,
 email          VARCHAR(80) UNIQUE,
 license_number VARCHAR(20) NOT NULL UNIQUE,
 license_expiry DATE NOT NULL,
 address        VARCHAR(120)
) ENGINE=InnoDB;


-- ---------------------------- transaction tables
CREATE TABLE Reservation (
 reservation_id INT AUTO_INCREMENT PRIMARY KEY,
 customer_id    INT NOT NULL,
 vehicle_id     INT NOT NULL,
 branch_id      INT NOT NULL,
 rate_plan_id   INT NOT NULL,
 booking_date   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
 planned_pickup DATETIME NOT NULL,
 planned_return DATETIME NOT NULL,
 status ENUM('BOOKED','ISSUED','COMPLETED','CANCELLED')
        NOT NULL DEFAULT 'BOOKED',
 CONSTRAINT chk_res_dates CHECK (planned_return > planned_pickup),
 FOREIGN KEY (customer_id)  REFERENCES Customer(customer_id),
 FOREIGN KEY (vehicle_id)   REFERENCES Vehicle(vehicle_id),
 FOREIGN KEY (branch_id)    REFERENCES Branch(branch_id),
 FOREIGN KEY (rate_plan_id) REFERENCES RatePlan(rate_plan_id)
) ENGINE=InnoDB;


CREATE TABLE Rental (
 rental_id         INT AUTO_INCREMENT PRIMARY KEY,
 reservation_id    INT NOT NULL UNIQUE,
 actual_pickup_dt  DATETIME,
 actual_return_dt  DATETIME,
 pickup_odometer   INT,
 return_odometer   INT,
 status ENUM('OPEN','OVERDUE','RETURNED') NOT NULL DEFAULT 'OPEN',
 CONSTRAINT chk_rental_dates
   CHECK (actual_return_dt IS NULL OR actual_return_dt >
actual_pickup_dt),
 CONSTRAINT chk_rental_odometer
   CHECK (return_odometer IS NULL OR return_odometer >= pickup_odometer),
 FOREIGN KEY (reservation_id) REFERENCES Reservation(reservation_id)
) ENGINE=InnoDB;
```

```
CREATE TABLE Inspection (
 inspection_id   INT AUTO_INCREMENT PRIMARY KEY,
 rental_id       INT NOT NULL,
 inspection_type ENUM('PICKUP','RETURN') NOT NULL,
 odometer_reading INT NOT NULL,
 fuel_level_pct  INT NOT NULL,
 condition_notes VARCHAR(200),
 inspected_by    VARCHAR(40),
 CONSTRAINT chk_inspection_fuel CHECK (fuel_level_pct BETWEEN 0 AND 100),
 CONSTRAINT uq_inspection_type_per_rental UNIQUE (rental_id,
inspection_type),
 FOREIGN KEY (rental_id) REFERENCES Rental(rental_id)
) ENGINE=InnoDB;


CREATE TABLE FuelReading (
 fuel_reading_id INT AUTO_INCREMENT PRIMARY KEY,
 inspection_id   INT NOT NULL UNIQUE,
 fuel_level_pct  INT NOT NULL,
 CONSTRAINT chk_fuel_pct CHECK (fuel_level_pct BETWEEN 0 AND 100),
 FOREIGN KEY (inspection_id) REFERENCES Inspection(inspection_id)
) ENGINE=InnoDB;


CREATE TABLE Damage (
 damage_id     INT AUTO_INCREMENT PRIMARY KEY,
 inspection_id INT NOT NULL,
 description   VARCHAR(200) NOT NULL,
 repair_cost   DECIMAL(10,2) NOT NULL,
 CONSTRAINT chk_damage_cost CHECK (repair_cost >= 0),
 FOREIGN KEY (inspection_id) REFERENCES Inspection(inspection_id)
) ENGINE=InnoDB;


CREATE TABLE Invoice (
 invoice_id       INT AUTO_INCREMENT PRIMARY KEY,
 rental_id        INT NOT NULL UNIQUE,
 rental_charges   DECIMAL(12,2) NOT NULL,
 extra_km_charges DECIMAL(12,2) NOT NULL DEFAULT 0,
 late_fee         DECIMAL(12,2) NOT NULL DEFAULT 0,
 damage_charges   DECIMAL(12,2) NOT NULL DEFAULT 0,
 tax_amount       DECIMAL(12,2) NOT NULL,
 total_amount     DECIMAL(12,2) NOT NULL,
 status ENUM('UNPAID','PARTIAL','PAID') NOT NULL DEFAULT 'UNPAID',
 FOREIGN KEY (rental_id) REFERENCES Rental(rental_id)
) ENGINE=InnoDB;


CREATE TABLE Payment (
 payment_id   INT AUTO_INCREMENT PRIMARY KEY,
 invoice_id   INT NOT NULL,
 amount       DECIMAL(12,2) NOT NULL,
 payment_mode ENUM('CASH','CARD','UPI','NETBANKING') NOT NULL,
 payment_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT chk_payment_amount CHECK (amount > 0),
 FOREIGN KEY (invoice_id) REFERENCES Invoice(invoice_id)
) ENGINE=InnoDB;


-- ---------------------------- triggers
DELIMITER $$


CREATE TRIGGER trg_no_overlap_reservation
BEFORE INSERT ON Reservation FOR EACH ROW
BEGIN
 IF EXISTS (SELECT 1 FROM Reservation r
            WHERE r.vehicle_id = NEW.vehicle_id
              AND r.status IN ('BOOKED','ISSUED')
              AND NEW.planned_pickup < r.planned_return
              AND NEW.planned_return > r.planned_pickup) THEN
   SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT =
     'Overlapping reservation exists for this vehicle in the selected
period.';
 END IF;
END$$
```

```
CREATE TRIGGER trg_valid_license
BEFORE INSERT ON Reservation FOR EACH ROW
BEGIN
 IF (SELECT license_expiry FROM Customer
     WHERE customer_id = NEW.customer_id) < DATE(NEW.planned_pickup) THEN
   SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT =
     'Customer driving licence is expired before the pickup date.';
 END IF;
END$$


CREATE TRIGGER trg_vehicle_status_on_pickup
AFTER UPDATE ON Rental FOR EACH ROW
BEGIN
 IF OLD.actual_pickup_dt IS NULL AND NEW.actual_pickup_dt IS NOT NULL
THEN
   UPDATE Vehicle SET status = 'RENTED'
   WHERE vehicle_id = (SELECT vehicle_id FROM Reservation
                       WHERE reservation_id = NEW.reservation_id);
 END IF;
 IF OLD.actual_return_dt IS NULL AND NEW.actual_return_dt IS NOT NULL
THEN
   UPDATE Vehicle SET status = 'AVAILABLE', odometer_km =
NEW.return_odometer
   WHERE vehicle_id = (SELECT vehicle_id FROM Reservation
                       WHERE reservation_id = NEW.reservation_id);
 END IF;
END$$


DELIMITER ;


-- ---------------------------- reporting views
CREATE VIEW vw_fleet_availability AS
SELECT b.city, b.branch_name, vc.category_name, v.vehicle_id,
      v.registration_no, v.model_name, v.status
FROM Vehicle v
JOIN VehicleCategory vc ON vc.category_id = v.category_id
JOIN Branch b ON b.branch_id = v.branch_id;


CREATE VIEW vw_overdue_rentals AS
SELECT rt.rental_id, c.full_name, v.registration_no, v.model_name,
      r.planned_return,
      TIMESTAMPDIFF(HOUR, r.planned_return, NOW()) AS hours_overdue
FROM Rental rt
JOIN Reservation r ON r.reservation_id = rt.reservation_id
JOIN Customer c ON c.customer_id = r.customer_id
JOIN Vehicle v ON v.vehicle_id = r.vehicle_id
WHERE rt.actual_pickup_dt IS NOT NULL
 AND rt.actual_return_dt IS NULL
 AND r.planned_return < NOW();


CREATE VIEW vw_vehicle_utilization AS
SELECT v.vehicle_id, v.registration_no, v.model_name,
      COUNT(rt.rental_id) AS rentals,
      COALESCE(SUM(rt.return_odometer - rt.pickup_odometer), 0) AS
km_driven
FROM Vehicle v
LEFT JOIN Reservation r ON r.vehicle_id = v.vehicle_id
LEFT JOIN Rental rt ON rt.reservation_id = r.reservation_id
GROUP BY v.vehicle_id, v.registration_no, v.model_name;


CREATE VIEW vw_damage_report AS
SELECT d.damage_id, v.registration_no, v.model_name, c.full_name,
      d.description, d.repair_cost
FROM Damage d
JOIN Inspection i ON i.inspection_id = d.inspection_id
JOIN Rental rt ON rt.rental_id = i.rental_id
JOIN Reservation r ON r.reservation_id = rt.reservation_id
JOIN Vehicle v ON v.vehicle_id = r.vehicle_id
JOIN Customer c ON c.customer_id = r.customer_id;


CREATE VIEW vw_branch_revenue AS
SELECT b.branch_id, b.branch_name, b.city,
      COUNT(inv.invoice_id) AS invoices,
```

```
      COALESCE(SUM(inv.total_amount), 0) AS billed,
      COALESCE(SUM(p.paid), 0) AS collected
FROM Branch b
LEFT JOIN Reservation r ON r.branch_id = b.branch_id
LEFT JOIN Rental rt ON rt.reservation_id = r.reservation_id
LEFT JOIN Invoice inv ON inv.rental_id = rt.rental_id
LEFT JOIN (SELECT invoice_id, SUM(amount) AS paid
          FROM Payment GROUP BY invoice_id) p ON p.invoice_id =
inv.invoice_id
GROUP BY b.branch_id, b.branch_name, b.city;


CREATE VIEW vw_customer_history AS
SELECT c.customer_id, c.full_name, r.reservation_id, v.model_name,
      r.planned_pickup, r.planned_return, r.status, inv.total_amount
FROM Customer c
JOIN Reservation r ON r.customer_id = c.customer_id
JOIN Vehicle v ON v.vehicle_id = r.vehicle_id
LEFT JOIN Rental rt ON rt.reservation_id = r.reservation_id
LEFT JOIN Invoice inv ON inv.rental_id = rt.rental_id;


-- ---------------------------- sample data (excerpt)
INSERT INTO Branch (branch_name, city) VALUES
 ('Woxsen Mumbai', 'Mumbai'), ('Woxsen Bengaluru', 'Bengaluru'), ('Woxsen
Pune', 'Pune');
INSERT INTO VehicleCategory (category_name, seating_capacity,
base_daily_rate) VALUES
 ('Hatchback', 5, 1800), ('Sedan', 5, 2500), ('SUV', 7, 3200), ('Luxury',
5, 6500);
INSERT INTO Customer (full_name, phone, email, license_number,
license_expiry) VALUES
 ('Rohan Mehta', '9000000001', 'rohan@example.com', 'LIC0001', '2027-05-
10'),
 ('Ananya Rao',  '9000000002', 'ananya@example.com', 'LIC0002', '2026-11-
20'),
 ('Karan Verma', '9000000003', 'karan@example.com', 'LIC0003', '2025-01-
15');
```

APPENDIX B: SOURCE CODE REFERENCE

The source code is organised as a Flask back end and a React front end. The back-end files and their responsibilities are listed below.

- `app.py`: creates the Flask application, registers the three blueprints under `/api`, serialises DECIMAL and DATETIME values to JSON, and installs error handlers.
- `db.py`: connection settings, the `transaction()` context manager and MySQL error translation.
- `validators.py`: input parsers for text, integers, money, dates, choices, phone and email.
- `billing.py`: invoice calculation using decimal arithmetic.
- `routes_catalog.py`: vehicles, customers, categories, branches, rate plans and statistics.
- `routes_operations.py`: availability, reservations, issue, return, invoices and payments.
- `routes_reports.py`: the six reporting views.
- `requirements.txt`: Flask 3.0.3, flask-cors 5.0.0, python-dotenv 1.0.1, mysql-connector-python 9.0.0, gunicorn 22.0.0.

The front end is a Vite project (`package.json`, `index.html`) using React 19, react-router-dom, gsap, lenis, motion and Phosphor icons. It is started with `npm run dev` and the API with `python app.py`, which serves on port 5050.